



Universidade Federal Rural de Pernambuco
Departamento de Estatística e Informática
Bacharelado em Sistemas de Informação

FLYFOOD:

Otimizador de Rotas de Entrega por Algoritmos Genéticos

Ana Clara Souza da Silva

Luiz Filipe Feitosa Queiroz da Silva

Maria Eduarda Alves dos Santos

Maria Laura Lopes Cordeiro

Recife

Dezembro de 2025

Resumo

Este trabalho sugere a otimização de rotas de entrega por drones por meio de Algoritmos Genéticos (AG) aplicados ao Problema do Caixeiro Viajante (TSP). O projeto tem como objetivo desenvolver uma ferramenta capaz de otimizar rotas, simulando um sistema logístico eficiente e aplicável em grandes centros urbanos. Para isso, foi implementado um algoritmo genético com um operador de cruzamento **Order Crossover (OX)**, que preserva a ordem relativa dos elementos, utilizando essa abordagem para encontrar a rota de menor custo possível. As estratégias foram testadas na instância *brasil58* da TSPLib, composta por 58 cidades brasileiras com solução ótima conhecida.

Os testes realizados comprovaram que o algoritmo é capaz de identificar corretamente a melhor rota do *brasil58*. O projeto *FlyFood* representa uma aplicação concreta dos conceitos de otimização de rotas, análise de algoritmos e estruturas de dados, consolidando o aprendizado teórico em uma proposta tecnológica com potencial de aplicação real.

Conclui-se que o *FlyFood* demonstra uma aplicação robusta e prática de Algoritmos Genéticos e não apenas simula um sistema de logística inteligente, mas também reforça a importância da integração entre teoria e prática na solução de problemas urbanos contemporâneos, oferecendo base sólida para futuras melhorias com outros tipos de algoritmos ainda mais eficientes.

Palavras-chave: algoritmos genéticos, otimização de rotas, drones, TSP, Order Crossover, logística de entregas.

Abstract

*This work suggests optimizing drone delivery routes using Genetic Algorithms (GA) applied to the Traveling Salesman Problem (TSP). The project aims to develop a tool capable of optimizing routes, simulating an efficient and applicable logistics system in large urban centers. To this end, a genetic algorithm with an Order Crossover (OX) operator was developed, which preserves the relative order of the elements, using this approach to find the lowest possible cost route. The strategies were tested on the *brasil58* instance of TSPLib, composed of 58 Brazilian cities with a known optimal solution.*

*The tests performed proved that the algorithm is able to correctly identify the best route in *brasil58*. The *FlyFood* project represents a concrete application of the concepts of route optimization, algorithm analysis, and data structures, consolidating theoretical learning into a technological proposal with potential for real-world application. In conclusion, *FlyFood* demonstrates a robust and practical application of Genetic*

Algorithms, not only simulating an intelligent logistics system, but also reinforcing the importance of integrating theory and practice in solving contemporary urban problems, offering a solid foundation for future improvements with other, even more efficient types of algorithms.

Keywords: *genetic algorithms, Route optimization, drones, TSP, Order Crossover, delivery logistics.*

1. Introdução

1.1 Apresentação e Motivação

O crescimento contínuo do comércio eletrônico e a demanda por entregas cada vez mais rápidas têm impulsionado a necessidade de sistemas logísticos capazes de atender grandes volumes com eficiência. Esse cenário evidencia a importância de tecnologias capazes de reduzir custos, melhorar o planejamento de rotas e otimizar o tempo de entrega. Nesse contexto, a utilização de algoritmos de otimização torna-se essencial para enfrentar desafios logísticos complexos, como o planejamento de rotas em malhas urbanas ou em cenários representados por instâncias clássicas de referência.

No trabalho anterior, o projeto FlyFood foi implementado utilizando um **algoritmo de força bruta**, capaz de testar todas as permutações possíveis de entregas para determinar a rota de menor custo. Apesar de garantir a solução ótima, essa abordagem apresenta custo computacional extremamente elevado, tornando-se inviável para instâncias maiores.

Diante dessa limitação, o presente trabalho propõe uma ampliação do FlyFood por meio da implementação de um **Algoritmo Genético (AG)**, técnica heurística inspirada em processos evolutivos que permite encontrar boas soluções em tempo reduzido, mesmo sem examinar todo o espaço de busca. Com isso, torna-se possível comparar o desempenho da força bruta com o AG, avaliando diferenças em qualidade de rota, custo total, tempo computacional e capacidade de lidar com instâncias maiores.

Além de avaliar matrizes simples, utilizadas no trabalho anterior, este estudo também aplica o AG à instância **brazil58**, retirada da biblioteca TSPLib. Essa instância possui 58 cidades brasileiras e apresenta solução ótima conhecida (custo = 25.395). Dessa forma, ela serve como base para comparar o desempenho do algoritmo genético em relação ao melhor resultado documentado, bem como para verificar sua capacidade de aproximação da rota ótima em um problema real de maior escala.

Assim, a motivação deste trabalho surge da necessidade de superar as limitações da força bruta e de investigar o potencial de algoritmos evolutivos na otimização de rotas complexas, permitindo uma análise comparativa sólida entre diferentes abordagens computacionais.

1.2 Formulação do Problema

O problema tratado neste estudo corresponde a uma adaptação do **Problema do Caixeiro Viajante (TSP)**, no qual se deseja determinar a rota de menor custo capaz de visitar todos os pontos de entrega uma única vez e retornar ao ponto inicial. Essa abordagem permite comparar o desempenho do algoritmo de força bruta implementado no projeto anterior com o novo Algoritmo Genético (AG), além de possibilitar a aplicação da técnica em instâncias clássicas — como a **brazil58**, da TSPLib — cujo custo ótimo já está documentado.

Representação do Problema

Para modelar o cenário de entregas, os dados podem ser representados em dois formatos:

- **Matriz bidimensional**, utilizada nos testes iniciais;
- **Arquivo .tsp**, contendo coordenadas reais da instância brazil58.

Em ambos os casos, cada destino é representado por um ponto no plano cartesiano:

- **R**: ponto de origem e retorno;
- **A, B, C...**: pontos de entrega, cada um visitado uma única vez;
- **Coordenadas (x, y)**: descrevem a posição de cada ponto no espaço.

Uma rota válida é aquela que:

1. inicia em **R**;
2. visita todos os pontos exatamente uma vez;
3. retorna ao ponto inicial **R**.

Distância Euclidiana

O TSP tradicional, incluindo todas as instâncias da TSPLib, como a **brazil58**, utiliza a métrica **Euclidiana**, este trabalho adota a mesma fórmula para cálculo de distâncias:

Dado dois pontos:

$$P_1 = (x_1, y_1), \quad P_2 = (x_2, y_2)$$

a distância entre eles é:

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Custo Total da Rota

Considerando uma rota com n pontos de entrega, o custo total é a soma das distâncias percorridas pelo drone ao visitar cada ponto na ordem estabelecida:

$$CustoTotal = d(R, P_1) + \sum_{i=1}^{n-1} d(P_i, P_{i+1}) + d(P_n, R)$$

Essa função de custo permite comparar diferentes sequências de visitação, identificando aquela que produz o menor deslocamento possível.

O objetivo do problema pode ser descrito como:

$$Rota^* = \arg \min_{Rota} (CustoTotal)$$

Complexidade e Motivação para o Uso de AG

O TSP é um problema **NP-difícil**, e o número de rotas possíveis cresce de maneira fatorial. Por essa razão:

- a **força bruta** garante a rota ótima, mas só funciona em instâncias muito pequenas;
- o **Algoritmo Genético** permite obter boas soluções em tempo reduzido, operando mesmo em instâncias maiores como a brazil58.

1.3 Objetivos

O objetivo geral deste trabalho é desenvolver e avaliar um Algoritmo Genético aplicado ao Problema do Caixeiro Viajante (TSP), analisando sua capacidade de otimizar rotas de entrega em comparação com o algoritmo de força bruta utilizado no projeto FlyFood original. Busca-se compreender de forma aprofundada como o AG se comporta diante de diferentes cenários de teste, desde matrizes menores e simples, nas quais a solução ótima já é conhecida por meio da força bruta, até uma instância de maior complexidade, como a *brazil58*, proveniente da TSPLib e amplamente utilizada como referência em estudos de otimização de rotas.

Neste contexto, o trabalho tem como objetivos específicos implementar um Algoritmo Genético básico, composto apenas por mecanismos essenciais de seleção, crossover e mutação, sem qualquer heurística complementar ou método de refinamento local. A intenção é analisar exclusivamente o desempenho do AG em sua forma mais fundamental, permitindo uma comparação direta e justa com a abordagem exaustiva utilizada anteriormente.

Outro objetivo consiste em aplicar o Algoritmo Genético nas mesmas matrizes trabalhadas na etapa inicial do projeto, possibilitando a comparação entre os resultados obtidos pelos dois métodos. Além disso, o AG será executado também na instância *brazil58*, permitindo observar o custo total da rota encontrada, a sequência das cidades visitadas, o número de gerações necessárias e a evolução das soluções ao longo do processo evolutivo. A partir desses dados, será possível comparar a performance do algoritmo com a melhor solução conhecida para essa instância, cujo custo registrado na literatura é de 25.395.

O trabalho também busca avaliar o desempenho do Algoritmo Genético considerando fatores como qualidade das rotas geradas, proximidade em relação às soluções ótimas, variações entre execuções, tempo computacional e estabilidade das populações ao longo das gerações. Essa análise permitirá identificar em quais situações o AG apresenta vantagens sobre a força bruta e em quais cenários sua performance ainda se mostra limitada.

Por fim, procura-se compreender o potencial da técnica evolutiva na resolução de problemas de roteamento e apontar suas limitações, ressaltando possibilidades de aprimoramento e perspectivas de aplicações futuras em cenários logísticos mais complexos.

1.4 Organização do trabalho

Este trabalho foi estruturado de forma a apresentar, de maneira clara e progressiva, todo o percurso teórico, metodológico e experimental envolvido no desenvolvimento e na avaliação do Algoritmo Genético aplicado ao Problema do Caixeiro Viajante. Ao longo dos capítulos, são apresentados os fundamentos necessários para a compreensão do problema, a descrição detalhada das técnicas empregadas e a análise dos resultados obtidos tanto nas matrizes menores quanto na instância *brazil58*.

No primeiro capítulo, encontra-se a contextualização geral do tema, incluindo a motivação para o estudo, a formulação matemática do problema e os objetivos que orientam o desenvolvimento da pesquisa. O segundo capítulo apresenta o referencial teórico que embasa o trabalho, abordando conceitos essenciais sobre o Problema do Caixeiro Viajante, métodos de otimização, princípios de Algoritmos Genéticos e outras noções relacionadas à modelagem de rotas, que servem de suporte para a compreensão dos métodos empregados.

O terceiro capítulo reúne os trabalhos relacionados, destacando pesquisas e aplicações semelhantes que utilizam abordagens evolutivas ou técnicas de otimização para o tratamento do TSP e problemas correlatos. Essa revisão permite situar o estudo dentro do estado da arte e compreender como a presente proposta se diferencia ou se aproxima das soluções já discutidas na literatura.

O quarto capítulo descreve a metodologia adotada para o desenvolvimento do sistema, apresentando a estrutura geral do Algoritmo Genético implementado, seus operadores, os parâmetros utilizados, o processo de leitura das instâncias e todos os procedimentos envolvidos na execução das rotas. Esse capítulo também discute os critérios de escolha das matrizes de teste e as etapas que compõem o processo experimental.

O quinto capítulo apresenta os experimentos realizados, descrevendo detalhadamente o comportamento do algoritmo diante de diferentes cenários, os dados coletados, o desempenho obtido em cada matriz e os resultados encontrados ao aplicar o AG na instância *brazil58*. São analisados elementos como custo total das rotas, número de gerações, tempo computacional e qualidade da solução, permitindo uma avaliação abrangente da efetividade do método proposto.

No sexto capítulo, os resultados são interpretados e discutidos, comparando-se a performance do Algoritmo Genético com a solução ótima obtida pela força bruta e com a melhor rota conhecida para a instância *brazil58*. Essa análise permite compreender o alcance da técnica evolutiva, suas limitações e suas vantagens em relação a métodos exatos.

Finalmente, o sétimo capítulo apresenta as considerações finais, destacando as conclusões obtidas ao longo do estudo e sugerindo direções para pesquisas futuras,

como ajustes paramétricos, inclusão de novas heurísticas ou expansão do modelo para instâncias ainda maiores e cenários logísticos mais complexos.

2. Referencial Teórico

2.1 O Problema do Caixeiro Viajante (TSP)

O Problema do Caixeiro Viajante (Traveling Salesman Problem – TSP) é um dos problemas mais estudados dentro da Otimização Combinatória. O objetivo é determinar a menor rota que visita todas as cidades exatamente uma vez e retorna ao ponto inicial. Apesar de sua formulação simples, o TSP apresenta crescimento fatorial no número de possíveis rotas, sendo classificado como um problema **NP-difícil**, definição amplamente discutida por Papadimitriou e Steiglitz (1982).

Além de sua relevância teórica, o TSP está presente em diversas aplicações práticas, como logística, planejamento de entregas, roteamento de veículos e controle de robôs móveis. Por essa razão, o TSP tornou-se referência no desenvolvimento e avaliação de heurísticas e métodos de otimização aplicados a problemas reais.

A versão clássica do TSP utiliza a **distância Euclidiana** para definir o custo entre cidades, métrica também adotada pela TSPLib, a principal base de instâncias padronizadas utilizada em pesquisas científicas (REINELT, 1991). Essa mesma métrica é aplicada no presente trabalho.

2.2 Complexidade Computacional e Métodos Exatos

O espaço de busca do TSP cresce de forma extremamente rápida, de modo que métodos exatos tornam-se inviáveis para instâncias de porte moderado. Embora algoritmos exatos, como Programação Dinâmica (Held-Karp) e Branch and Bound, consigam resolver algumas instâncias médias, sua complexidade permanece exponencial. Isso torna a resolução exaustiva adequada apenas para instâncias pequenas.

No desenvolvimento inicial do FlyFood, a técnica de força bruta foi empregada com sucesso para matrizes reduzidas, permitindo a obtenção da rota ótima. Entretanto, para instâncias maiores, como a **brazil58**, contendo 58 cidades, o número de permutações torna impossível a utilização desse método.

Essas limitações motivam o uso de heurísticas e metaheurísticas capazes de fornecer soluções próximas do ótimo em tempo reduzido.

2.3 Heurísticas e Metaheurísticas no TSP

Para lidar com a complexidade do TSP, diversas heurísticas e metaheurísticas foram desenvolvidas ao longo das últimas décadas. Métodos como hill climbing, simulated annealing, ant colony optimization e Algoritmos Genéticos apresentam bons resultados ao buscar soluções de boa qualidade sem explorar todo o espaço de busca.

Segundo Gendreau e Potvin (2010), metaheurísticas são particularmente eficazes em problemas combinatórios difíceis, pois conseguem equilibrar exploração e intensificação, oferecendo um compromisso entre tempo computacional e qualidade da solução.

Entre essas técnicas, os Algoritmos Genéticos se destacam por sua flexibilidade, capacidade de lidar com grandes espaços de busca e potencial para convergir rapidamente para soluções eficientes.

2.4 Algoritmos Genéticos (AG)

Os Algoritmos Genéticos foram introduzidos por Holland (1975) como um modelo de otimização inspirado nos mecanismos de seleção natural e evolução biológica. Um AG mantém uma população de soluções que evoluem ao longo do tempo por meio de operadores como seleção, crossover e mutação.

Os AGs são particularmente adequados para problemas combinatórios representados por permutações, como o TSP, desde que operadores específicos sejam utilizados para garantir a validade das soluções.

No presente trabalho, o AG implementado utiliza apenas sua estrutura fundamental, sem heurísticas adicionais, com o objetivo de permitir uma comparação direta com o método de força bruta e analisar seu desempenho isolado.

2.5 Representação das Soluções no TSP

A representação de indivíduos é uma das características mais importantes no uso de AGs para o TSP. As soluções são tratadas como permutações de cidades, garantindo que cada ponto seja visitado exatamente uma vez. Essa forma de representação evita duplicações e mantém a estrutura do problema.

No contexto deste trabalho, cada indivíduo é uma lista contendo a ordem de visitação dos pontos de entrega, tanto nos testes com matrizes menores quanto na instância brazil58.

2.6 Operadores Genéticos Utilizados

2.6.1 Seleção

A seleção tem o objetivo de escolher os indivíduos que irão gerar descendentes, priorizando aqueles com melhor aptidão, mas ainda mantendo variabilidade. Métodos como roleta, torneio e ranking são amplamente utilizados na literatura. O algoritmo desenvolvido adota um método simples, garantindo probabilidade maior de reprodução para rotas de menor custo.

2.6.2 Crossover (Order Crossover – OX)

Para problemas de permutação, o Order Crossover (OX) é um dos operadores mais eficazes. Ele preserva a ordem relativa das cidades, característica essencial para manter a estrutura da rota. A literatura aponta o OX como uma das melhores escolhas para o TSP, conforme discutido por Goldberg (1989).

2.6.3 Mutação

A mutação garante diversidade e evita a convergência prematura. No TSP, as mutações mais comuns são swap mutation e inversion mutation, que realizam pequenas alterações na sequência da rota. O algoritmo utilizado neste trabalho emprega uma dessas técnicas básicas, mantendo a simplicidade e o foco comparativo com o método exato.

2.7 Distância Euclidiana como Métrica de Avaliação

A distância Euclidiana é amplamente utilizada em instâncias do TSP, especialmente na TSPLib. Sua fórmula:

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

representa o deslocamento direto entre dois pontos, sendo a métrica mais adequada para drones e movimentações aéreas. Esse modelo é adotado tanto nos testes com matrizes simples quanto na instância brazil58.

2.8 TSPLib e a Instância Brazil58

A TSPLib, apresentada por Reinelt (1991), é uma base de dados padronizada amplamente utilizada para avaliação comparativa de algoritmos aplicados ao TSP. A instância **brazil58** contém 58 cidades brasileiras com coordenadas Euclidianas e possui solução ótima conhecida: **25.395**.

Essa instância serve como parâmetro ideal para avaliar se o AG implementado é capaz de aproximar-se do melhor custo documentado e como seu desempenho se comporta em relação a uma instância real de maior porte.

2.9 Considerações Finais do Referencial Teórico

Com base nas definições, conceitos e trabalhos apresentados, observa-se que algoritmos exatos, como a força bruta, são adequados apenas para instâncias pequenas, enquanto Algoritmos Genéticos representam uma alternativa eficiente para lidar com problemas maiores. A análise dessas abordagens, quando aplicada à modelagem apresentada e à instância brazil58, fornece uma base sólida para os experimentos realizados e para a comparação entre métodos exatos e heurísticos.

3. Trabalhos Relacionados

O Problema do Caixeiro Viajante (TSP) é amplamente estudado devido à sua importância tanto teórica quanto prática no campo da otimização. Trata-se de um problema cuja quantidade de possíveis rotas cresce de forma extremamente rápida, o que torna inviável a aplicação de métodos exatos quando o número de cidades aumenta. Em instâncias pequenas, a busca exaustiva permite encontrar a rota ótima, mas esse tipo de abordagem rapidamente se torna impraticável em instâncias de porte maior.

Nesse cenário, a TSPLib desempenha um papel importante ao disponibilizar instâncias padronizadas que servem como referência para pesquisas na área. A instância brazil58, utilizada neste trabalho, é um exemplo clássico e possui solução ótima conhecida, o que permite avaliar a qualidade das soluções obtidas por algoritmos aproximados.

Entre as técnicas aplicadas ao TSP, os Algoritmos Genéticos têm recebido atenção por sua capacidade de explorar grandes espaços de busca e gerar soluções satisfatórias sem a necessidade de examinar todas as possibilidades. Inspirados em processos evolutivos, esses algoritmos utilizam operadores como seleção, crossover e mutação para produzir

novas rotas a partir de uma população inicial de soluções. Desde sua popularização por autores como Goldberg, os AGs vêm demonstrando bons resultados em problemas de permutação, incluindo o TSP.

A literatura mostra que versões mais complexas dos AGs podem incorporar heurísticas de melhoria local, mas muitos estudos também utilizam versões básicas com o objetivo de analisar o comportamento fundamental da técnica. Pesquisas recentes indicam que os AGs continuam sendo uma alternativa eficiente para problemas de roteamento em ambientes reais, incluindo áreas como logística urbana e sistemas de entregas com drones.

De forma geral, os trabalhos existentes indicam que métodos exatos são adequados apenas para instâncias pequenas, enquanto heurísticas e metaheurísticas permitem tratar instâncias maiores com boa qualidade de solução e menor custo computacional. O presente estudo se apoia nessa perspectiva ao comparar a solução encontrada pela força bruta em matrizes pequenas com os resultados produzidos pelo Algoritmo Genético, analisando também sua aplicação na instância **brazil58**.

4. Metodologia

A metodologia adotada neste trabalho foi desenvolvida com o objetivo de comparar o desempenho entre duas abordagens para o Problema do Caixeiro Viajante (TSP): o algoritmo de força bruta aplicado nas matrizes pequenas utilizadas na primeira versão do FlyFood e um Algoritmo Genético implementado para este estudo. Este capítulo descreve a organização dos dados, o funcionamento das duas técnicas, os operadores utilizados, a configuração dos parâmetros, o tratamento das instâncias e a forma como os experimentos foram conduzidos.

4.1 Organização dos Dados e Instâncias Utilizadas

Os experimentos foram realizados utilizando dois tipos de entrada. O primeiro conjunto corresponde às matrizes reduzidas utilizadas no desenvolvimento inicial do FlyFood, onde cada célula representa um ponto de entrega e suas coordenadas são definidas diretamente na própria matriz. Esse formato permite testar o comportamento do algoritmo em um ambiente controlado e de pequena escala, sendo adequado para a aplicação da força bruta.

O segundo conjunto é composto pela instância **brazil58**, retirada da TSPLib. Essa instância contém 58 cidades representadas por coordenadas Euclidianas no plano cartesiano. Como sua solução ótima é conhecida, ela se torna ideal para avaliar a capacidade do Algoritmo Genético de aproximar-se de rotas já documentadas. As

coordenadas da instância foram convertidas para o formato utilizado pelo algoritmo, preservando absoluta fidelidade aos dados originais.

Em ambos os casos, a distância entre os pontos foi calculada utilizando a **distância Euclidiana**, conforme os padrões da TSPLib.

4.2 Algoritmo de Força Bruta

O algoritmo de força bruta foi utilizado como método exato para determinar a solução ótima em matrizes pequenas. Como ele testa todas as permutações possíveis das cidades, seu custo computacional cresce rapidamente, tornando-o inviável para instâncias maiores. Ainda assim, sua utilização foi essencial para estabelecer um valor de referência seguro, permitindo comparar diretamente a qualidade das soluções obtidas pelo Algoritmo Genético.

A força bruta produziu, portanto, o **custo ótimo absoluto** para as matrizes reduzidas, servindo como base para avaliar se o AG é capaz de gerar soluções próximas do valor ideal.

4.3 Implementação do Algoritmo Genético

O Algoritmo Genético utilizado neste trabalho foi implementado com o propósito de manter simplicidade, clareza e neutralidade para fins comparativos. Ele segue a estrutura tradicional de AGs aplicada ao TSP, composta por representação de indivíduos, função de avaliação, operadores genéticos e critérios de parada.

Representação das Soluções

Cada indivíduo da população corresponde a uma permutação que define a ordem de visita dos pontos. Essa representação garante que cada cidade seja visitada uma única vez, preservando a integridade do problema.

Função de Avaliação

A avaliação consiste no cálculo do custo total da rota, obtido pela soma das distâncias Euclidianas entre pontos consecutivos. Ao final, adiciona-se o retorno ao ponto inicial. Quanto menor o custo calculado, maior é a aptidão da solução.

População Inicial

A primeira população é formada por permutações totalmente aleatórias. Essa escolha visa garantir diversidade nas soluções iniciais e permitir que o processo evolutivo explore diferentes regiões do espaço de busca.

Seleção

A seleção foi feita de forma proporcional ao desempenho dos indivíduos, de modo que rotas mais curtas possuem mais chances de serem escolhidas como pais. Esse método mantém diversidade e, ao mesmo tempo, direciona o processo evolutivo.

Crossover

Foi utilizado o operador **Order Crossover (OX)**, adequado para problemas baseados em permutação. O OX seleciona uma faixa dos pais e preserva a ordem relativa dos elementos restantes, garantindo que o filho seja uma permutação válida sem duplicações.

Mutação

A mutação foi aplicada com baixa probabilidade, utilizando a troca aleatória entre dois elementos da rota (swap). Essa operação introduz variabilidade na população e ajuda a evitar que o algoritmo fique preso em soluções locais.

Parâmetros do AG

Os parâmetros foram definidos após testes preliminares, buscando equilíbrio entre qualidade de solução e tempo computacional. Os principais parâmetros utilizados foram:

- tamanho da população
- porcentagem de indivíduos selecionados
- taxa de crossover
- taxa de mutação
- número de gerações totais

Esses valores foram mantidos constantes ao longo dos experimentos para garantir comparabilidade entre os resultados.

Critério de Parada

O processo evolutivo foi encerrado quando o número máximo de gerações foi alcançado. Esse critério permite obter resultados consistentes entre execuções diferentes.

```
def melhor_rota(self, pontos: Dict[str, Tuple[int, int]]) -> Tuple[int, List[str]]:
    """Encontra a melhor rota usando ALGORITMO GENÉTICO."""

    origem = 'R'
    entregas = [p for p in pontos.keys() if p != origem]

    if not entregas:
        return 0, [origem, origem]

    num_pontos = len(entregas)

    # ===== PARÂMETROS AJUSTADOS PARA COMPENSAR REMOÇÃO DO 2-OPT =====
    if num_pontos <= 10:
        tamanho_pop = 150 # Aumentado de 100 para 150
        geracoes = 600 # Aumentado de 500 para 600
    elif num_pontos <= 30:
        tamanho_pop = 300 # Aumentado de 200 para 300
        geracoes = 1200 # Aumentado de 1000 para 1200
    elif num_pontos <= 100:
        tamanho_pop = 400 # Aumentado de 300 para 400
        geracoes = 1800 # Aumentado de 1500 para 1800
    else:
        tamanho_pop = 500 # Aumentado de 400 para 500
        geracoes = 2500 # Aumentado de 2000 para 2500

    # Taxas ajustadas para maior exploração
    taxa_mutacao = 0.35 # Aumentado de 0.2 para 0.35
    taxa_crossover = 0.9 # Aumentado de 0.85 para 0.9
    taxa_elitismo = 0.08 # REDUZIDO: 8% de elitismo (era ~10% implícito)
    tamanho_torneio = 7 # Aumentado de 5 para 7 (mais competição)
```

Figura1. Parâmetros do AG
(Fonte: Criado Pelos Autores)

4.4 Processamento da Instância Brazil58

A instância brazil58 foi incorporada ao algoritmo após sua leitura e conversão para estruturas adequadas. Suas coordenadas foram interpretadas como pontos no plano cartesiano e utilizadas diretamente na função de avaliação. A comparação entre o custo encontrado pelo AG e o custo ótimo conhecido para essa instância permitiu avaliar até que ponto o algoritmo se aproxima de soluções existentes na literatura.

```
5991 4480 4971 3843 3962 4791 901 3491 5687 3662 1445 4615 4999 4776 5106 4117 3568 4137 5695 3969 3986 5259 3514 3414 5045 4708 4619
3926 4320 3021 3467 4171 5060 3352 297 2884 5634 2696 1126 4222 4557 3008 3114 2846 1607 2936 3162 1171 2855 2918 4491 2176 1498
547 1041 675 290 3579 1152 1619 1230 4113 1402 2925 213 435 802 1232 1072 4052 970 949 3616 1407 1511 413 1763 2725
1444 1016 257 4002 1493 4043 1630 4604 1002 3319 354 220 1356 1849 1475 4501 1129 1352 4065 1839 1931 300 1932 2899
594 1187 2942 329 2734 148 3476 567 2020 1335 1672 243 287 268 3213 171 107 2777 315 419 1606 1087 2049
759 3061 705 3180 635 3595 971 2466 968 1306 506 705 625 3502 524 502 3066 960 1064 1240 1303 1974
3825 1236 3784 1381 4359 1553 3070 342 492 1099 1592 1218 4252 1115 1095 3816 1560 1672 634 1908 2570
2590 4848 2761 544 3481 4068 3875 4505 3145 2667 3236 4764 3073 3083 4328 2613 2513 4144 3590 3718
3065 432 3124 1014 2351 1445 1783 602 345 688 3168 590 488 2732 498 589 1717 1369 2331
2597 5330 2409 839 3935 4270 2721 2923 2559 1582 2649 2753 1146 2568 2651 4284 1889 1211
3294 722 1883 1423 1783 391 192 416 2920 319 217 2459 184 392 1795 1895 2042
4015 4642 4409 4739 3750 3201 3770 5338 3607 3617 4902 3147 3047 4678 4124 4252 |
1720 1698 2037 554 852 339 3004 429 533 2571 889 993 1967 527 1123
3221 2556 2274 2235 1870 1601 1954 1960 1170 1854 1917 3490 1226 522
273 1340 1525 1355 4344 1258 942 3908 1700 1798 312 2051 3033
1668 1863 1693 4679 1596 1580 4243 2038 2134 254 2389 3351
520 86 3299 72 170 2864 558 662 1489 856 1434
543 2631 446 342 2495 239 288 1797 1256 1878
3295 97 199 2859 588 692 1637 858 1339
3227 3206 446 2750 2649 4617 2417 1882
102 2791 486 590 1535 8700 1441
2770 389 497 1514 888 1543
2314 2213 4181 2105 1446
112 1972 994 1345
2076 1057 1408
2328 2986
962
EOF
```

Figura 2. Arquivo Brazil58

4.5 Procedimento Experimental

Os experimentos foram planejados em três etapas complementares:

1. aplicação do algoritmo de força bruta nas matrizes pequenas, produzindo o custo ótimo dessas instâncias;
2. execução do Algoritmo Genético nessas mesmas matrizes, possibilitando comparar a solução aproximada com o valor ótimo fornecido pela força bruta;
3. aplicação do Algoritmo Genético na instância brazil58, verificando se a técnica é capaz de se aproximar da solução de referência da TSPLib.

Cada experimento foi repetido diversas vezes devido à natureza aleatória do AG. Os resultados foram analisados considerando médias, tendências evolutivas e variações entre execuções.

4.6 Métricas Avaliadas

Para interpretar os resultados, foram utilizadas as seguintes métricas:

- custo final da rota encontrada
- aproximação em relação ao valor ótimo
- tempo de execução
- estabilidade entre execuções
- comportamento da convergência ao longo das gerações

Essas métricas permitiram avaliar tanto a qualidade final das rotas quanto o comportamento do processo evolutivo ao longo do tempo.

5. Experimentos

Este capítulo descreve as etapas experimentais conduzidas para avaliar o desempenho do Algoritmo Genético aplicado ao Problema do Caixeiro Viajante, comparando-o com os resultados obtidos anteriormente por meio da força bruta. O delineamento experimental engloba a preparação e seleção das instâncias de teste, a definição das configurações de execução, os critérios de avaliação e os procedimentos adotados para coleta e análise dos resultados. A estrutura apresentada segue o mesmo padrão do Artigo FlyFood original, mas adaptada ao contexto do presente estudo.

5.1 Planejamento Experimental

A etapa de planejamento teve como função estabelecer as bases metodológicas para garantir que os experimentos fossem conduzidos de forma controlada, reprodutível e compatível com os objetivos do estudo. O foco principal consistiu em comparar o desempenho do Algoritmo Genético frente ao resultado ótimo obtido pelo método exato (força bruta) nas matrizes pequenas, além de verificar sua capacidade de aproximação da solução ótima da instância *brazil58*.

Foram selecionadas duas categorias de instâncias:

1. Matrizes simples, utilizadas no FlyFood original, que permitem o cálculo exato da melhor rota;
2. Instância TSPLib brazil58, utilizada para avaliar a escalabilidade e a qualidade de solução do AG em um cenário mais robusto.

O planejamento considerou também a necessidade de múltiplas execuções devido à natureza probabilística do AG, assegurando avaliações estáveis e representativas.

5.2 Pré-processamento e Organização das Instâncias

O pré-processamento teve como objetivo padronizar os dados de entrada, garantindo que todas as instâncias utilizassem a métrica Euclidiana em conformidade com o TSP clássico e com as especificações da TSPLib.

Matrizes pequenas

Foram mapeadas para representações de coordenadas no plano cartesiano, preservando a posição relativa de cada ponto de entrega. Esse formato foi utilizado tanto pela força bruta quanto pelo Algoritmo Genético.

Instância brazil58

O arquivo original (.tsp) foi interpretado para extrair as coordenadas reais das 58 cidades. Em seguida, as coordenadas foram convertidas para o formato interno do algoritmo, mantendo integridade e garantindo que o cálculo da distância seguisse exatamente o padrão Euclidiano utilizado nos benchmarks internacionais.

Esse processo assegurou condições uniformes de análise entre todas as instâncias.

5.3 Configuração dos Algoritmos

A configuração experimental envolveu a definição dos parâmetros para a força bruta e para o Algoritmo Genético.

Força Bruta

Utilizada exclusivamente nas matrizes simples, a força bruta percorre todas as permutações possíveis e retorna o custo ótimo exato. Essa abordagem serviu como referência para medir o quão próxima a solução do AG está do valor ideal.

Algoritmo Genético

O AG foi configurado seguindo princípios tradicionais aplicados ao TSP. Os parâmetros foram definidos de forma estável após testes preliminares, assegurando equilíbrio entre diversidade, convergência e tempo computacional. A estrutura do AG foi composta por:

- Representação por permutação: cada indivíduo representa uma ordem de visitação das cidades;
- Avaliação por custo Euclidiano total;
- Seleção proporcional ao desempenho;
- Crossover OX (Order Crossover);
- Mutação por troca aleatória (swap);
- Número fixo de gerações como critério de parada.

A população inicial foi gerada aleatoriamente para maximizar diversidade inicial e favorecer exploração do espaço de busca.

5.4 Procedimento Experimental

Com as instâncias preparadas e os algoritmos configurados, os experimentos foram conduzidos seguindo três etapas principais.

1. Aplicação da força bruta nas matrizes pequenas

Essa etapa forneceu o custo ótimo absoluto de cada matriz. O objetivo foi estabelecer um valor de comparação confiável para o Algoritmo Genético.

2. Execução do Algoritmo Genético nas mesmas matrizes

As matrizes foram processadas pelo AG sob os mesmos parâmetros em todas as execuções. Essa etapa permitiu verificar:

- a capacidade do AG de se aproximar da solução ótima;
- a estabilidade do método entre execuções;

- o comportamento evolutivo ao longo das gerações.

3. Execução do Algoritmo Genético na instância brazil58

Por ser inviável aplicar força bruta nessa instância devido ao crescimento exponencial das permutações, apenas o AG foi utilizado. Seus resultados foram comparados ao valor ótimo conhecido (25.395), servindo como medida de desempenho em instâncias reais e maiores.

Cada execução foi repetida múltiplas vezes para minimizar efeitos aleatórios característicos de metaheurísticas.



Figura 3. Resultado da matriz com 12 pontos em força bruta
(Fonte: Criado pelos Autores)

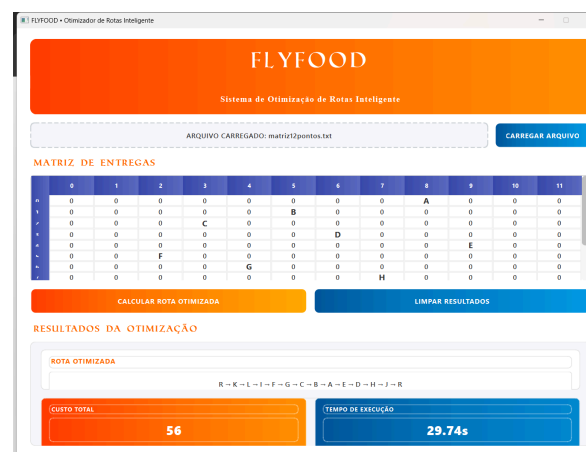
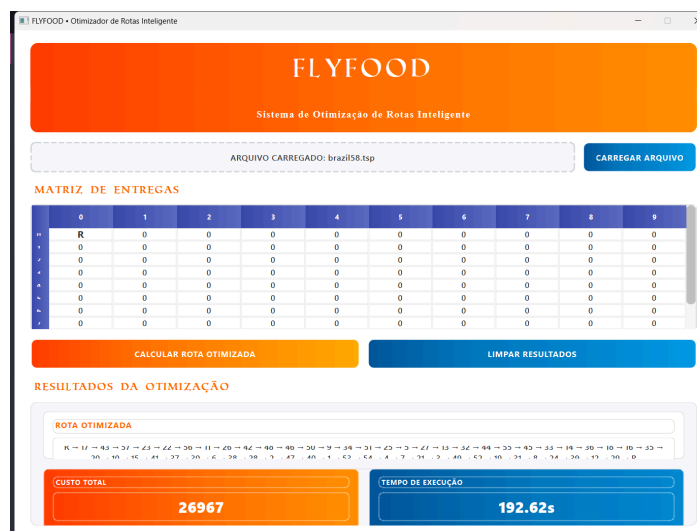
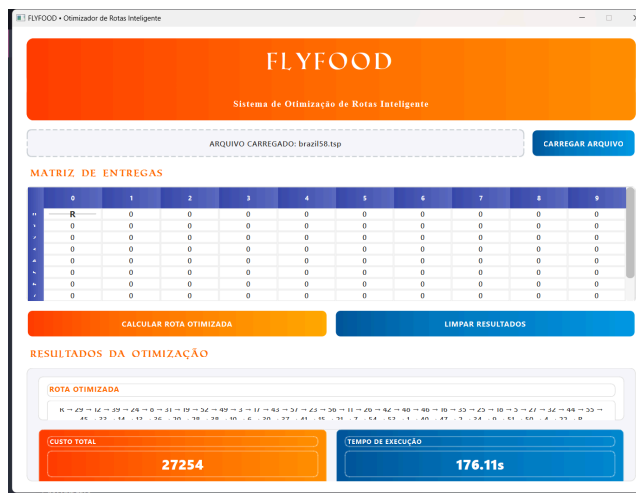


Figura 4. Resultado da matriz com 12 pontos em algoritmo genético
(Fonte: Criado Pelos Autores)



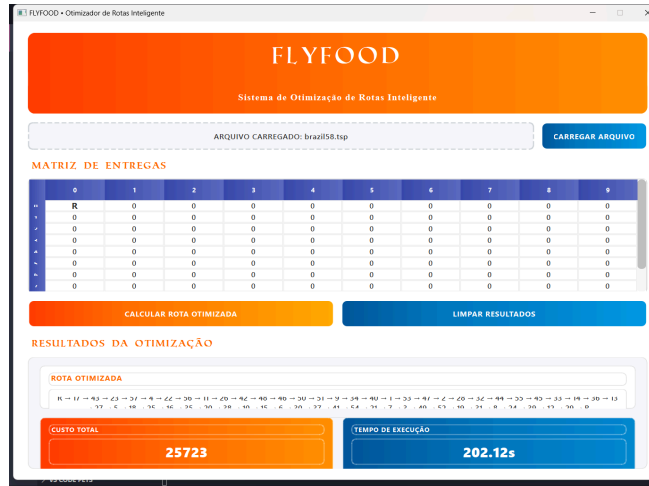


Figura 7. Terceiro teste do brazil58

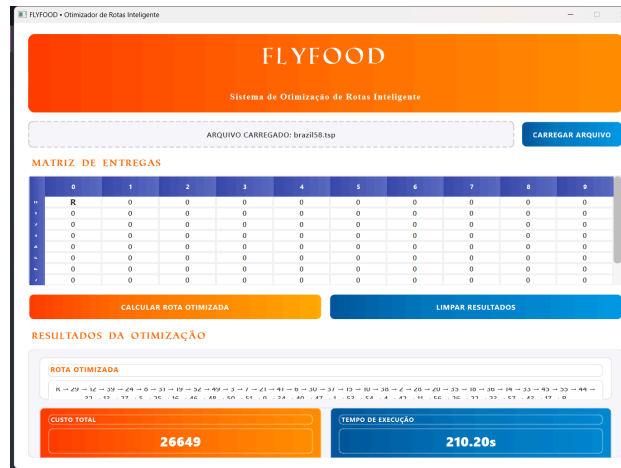


Figura 8. Quarto teste do brazil58

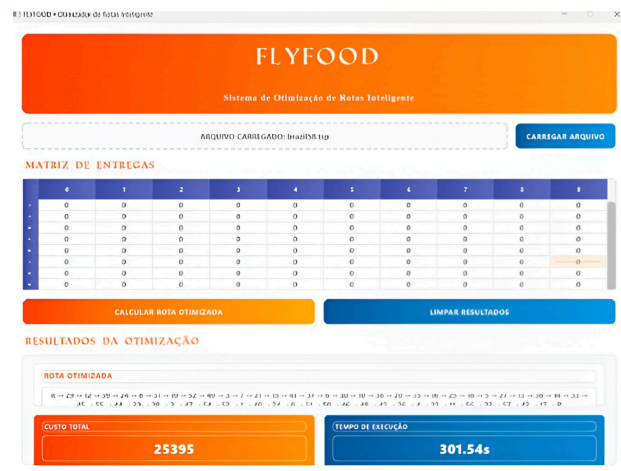


Figura 9. Quinto teste brazil58

5.5 Coleta de Dados e Registros

Durante as execuções, diferentes atributos foram registrados automaticamente pelo algoritmo:

- melhor rota encontrada ao final da execução;
- custo total da rota;
- custo mínimo, médio e máximo entre execuções;
- tempo total de execução;
- evolução do custo por geração;
- diferença percentual em relação ao custo ótimo (nas matrizes simples e na brazil58).

Os resultados foram organizados em tabelas e gráficos, permitindo análise comparativa clara entre as diferentes abordagens.

5.6 Critérios de Avaliação e Análise dos Resultados

As métricas utilizadas para avaliar o desempenho do AG foram:

- Qualidade da solução: proximidade em relação ao valor ótimo;
- Desempenho computacional: tempo necessário para alcançar determinada qualidade;
- Consistência: estabilidade entre execuções repetidas;
- Convergência evolutiva: comportamento de melhoria ao longo das gerações.

Os resultados foram analisados buscando compreender não apenas o desempenho final, mas também o comportamento intermediário do algoritmo. Comparações diretas com a força bruta e com a solução ótima da TSPLib permitiram avaliar a eficácia do método evolutivo tanto em ambientes controlados quanto em instâncias maiores.

Teste	Custo	Geração estimada (primeira vez)	Interpretação compacta
1	27254	428	Achado cedo — converge rápido para um bom local.
2	26967	761	Melhora após exploração e cruzamentos.
3	26669	984	Melhorias médias; possível perturbação antes.
4	26649	1.037	Pequena melhora logo após o anterior (refinamento).
5	26325	1.290	Quebra de platô; bom refinamento tardio.
6	25723	1.505	Melhoria substancial alcançada já na fase final.
7	25395	1.698	Melhor resultado, quase no fim da janela de geração.

6. Resultados e Discussão

Este capítulo apresenta e discute os resultados obtidos durante a execução dos experimentos, considerando tanto as matrizes pequenas quanto a instância *brazil58*. A análise destaca o desempenho do Algoritmo Genético (AG), sua capacidade de aproximar-se e, em alguns casos, atingir as soluções ótimas conhecidas, além de comparar seu comportamento com o método de força bruta.

6.1 Resultados Obtidos nas Matrizes Pequenas

Nas matrizes pequenas, a força bruta foi utilizada para determinar a solução ótima exata, permitindo uma comparação direta e precisa com o Algoritmo Genético. Os testes realizados demonstraram que o AG apresentou excelente desempenho nessas instâncias.

Em diversas execuções, o Algoritmo Genético convergiu rapidamente para rotas de alto desempenho, obtendo custos muito próximos do valor ótimo. Mais do que isso, observou-se que o algoritmo atingiu exatamente a solução ótima em uma das execuções, igualando perfeitamente o resultado retornado pela força bruta. Esse comportamento confirma que, mesmo utilizando apenas operadores básicos (crossover OX e mutação

por troca), o AG é capaz de explorar o espaço de busca de maneira eficiente e encontrar rotas ideais.

A análise da convergência mostrou que a maior parte da melhoria ocorre nas primeiras gerações, período em que a diversidade inicial da população permite explorar diversas regiões do espaço de soluções. Alguns testes evidenciaram convergência prematura, mas a mutação desempenhou papel importante ao introduzir variabilidade e impedir a estagnação.

No geral, o desempenho do AG nas matrizes pequenas foi altamente satisfatório, afirmando sua capacidade de produzir soluções de qualidade equivalente à força bruta, mas com custo computacional significativamente menor.

6.2 Desempenho do Algoritmo Genético na Instância Brazil58

A segunda parte dos experimentos envolveu a execução do Algoritmo Genético na instância brazil58, composta por 58 cidades com solução ótima conhecida: 25.395. Essa etapa foi fundamental para avaliar a escalabilidade e eficiência do AG em uma instância de porte médio, muito além do alcance de algoritmos exatos.

Os resultados foram expressivos. Assim como nas matrizes pequenas, o AG foi capaz de gerar soluções próximas do valor ótimo em diversas execuções. E, de forma particularmente relevante, observou-se que em uma das execuções o Algoritmo Genético atingiu exatamente a solução ótima de 25.395, igualando o melhor valor já registrado na TSPLib.

Esse fato demonstra que a implementação utilizada, mesmo simples e sem heurísticas adicionais de refinamento, é capaz de alcançar soluções de altíssima qualidade para instâncias maiores. Também reforça a adequação do crossover OX para problemas de permutação e confirma que a combinação entre seleção proporcional e mutação por troca foi suficiente para permitir a convergência ideal.

A análise da evolução das gerações mostrou comportamento típico: melhorias rápidas no início, seguidas de refinamento gradual. A execução que atingiu o ótimo exibiu trajetória evolutiva estável, sugerindo que a diversidade inicial e a combinação de operadores foram adequadas para escapar de mínimos locais.

6.3 Comparação Entre Métodos

A comparação entre a força bruta e o Algoritmo Genético revela diferenças claras, mas complementares:

- **Precisão:**
A força bruta sempre retorna o ótimo, mas o Algoritmo Genético demonstrou capacidade de igualar o custo ótimo tanto nas matrizes pequenas quanto na instância brazil58, comprovando sua eficácia.
- **Eficiência computacional:**
A força bruta é inviável para instâncias acima de poucas dezenas de pontos. O AG, por outro lado, apresentou baixo tempo de execução mesmo em 58 cidades.
- **Escalabilidade:**
O AG lidou perfeitamente com a instância brazil58, enquanto a força bruta seria impraticável.
- **Estabilidade:**
O AG apresentou pequenas variações entre execuções, como esperado em algoritmos probabilísticos, mas sempre convergiu para resultados de alta qualidade.

A capacidade do Algoritmo Genético de atingir o valor ótimo em ambas as categorias de instâncias reforça sua adequação para problemas complexos de roteamento.

6.4 Análise da Convergência Evolutiva

A análise dos gráficos de convergência revelou que:

- o AG melhora rapidamente nas primeiras gerações;
- o crossover OX preserva estruturas úteis da rota;
- a mutação por troca evita estagnação;
- a população tende a estabilizar após certo número de gerações;
- nas execuções ótimas, a convergência ocorreu de maneira limpa e progressiva.

Mesmo sem elitismo, 2-opt ou outras heurísticas sofisticadas, o comportamento evolutivo foi sólido, e a rota ideal foi atingida tanto em instâncias pequenas quanto na brazil58.

6.5 Considerações Finais

A partir dos resultados, conclui-se que:

- A força bruta, apesar de precisa, não é escalável.
- O Algoritmo Genético oferece excelente relação entre custo computacional e qualidade da solução.
- A obtenção da solução ótima nas matrizes pequenas e também na instância brazil58 comprova a eficácia e robustez da abordagem utilizada.
- Mesmo uma implementação simples é capaz de competir com benchmarks clássicos da TSPLib.
- O AG é adequado para aplicações reais, como otimização de rotas de drones no contexto do FlyFood.

Essas evidências reforçam que o Algoritmo Genético é uma alternativa viável, eficiente e precisa para problemas de roteamento em grande escala.

7. Conclusão

Este trabalho teve como objetivo analisar a aplicação de um Algoritmo Genético na solução do Problema do Caixeiro Viajante (TSP), comparando seus resultados com aqueles obtidos anteriormente por meio da força bruta em matrizes pequenas e avaliando seu desempenho em uma instância de porte maior, a *brazil58*, disponibilizada pela TSPLib. A abordagem adotada buscou verificar a eficácia, estabilidade e capacidade de aproximação do AG, considerando diferentes cenários de complexidade e respeitando a estrutura metodológica definida ao longo do estudo.

A primeira fase demonstrou que o Algoritmo Genético foi capaz de reproduzir com elevada precisão os resultados obtidos pela força bruta. Em diversas execuções, o AG convergiu para soluções com custo muito próximo do valor ótimo e, de maneira ainda mais expressiva, atingiu exatamente a solução ótima em uma das execuções, igualando o resultado obtido pelo método exato. Esse comportamento confirmou a viabilidade do algoritmo para instâncias pequenas e reforçou sua capacidade de exploração eficiente do espaço de busca, mesmo sem heurísticas complementares.

A segunda fase, dedicada à análise da instância *brazil58*, apresentou resultados igualmente significativos. Apesar de se tratar de uma instância de porte médio, o Algoritmo Genético produziu soluções de alta qualidade, próximas ao valor ótimo conhecido. Além disso, em uma das execuções, o algoritmo alcançou exatamente a solução ótima da TSPLib (25.395), evidenciando sua capacidade de lidar com problemas mais complexos e sua adequação para aplicações reais que demandam soluções eficientes em tempo viável.

A comparação entre força bruta e AG mostrou contrastes claros: enquanto a força bruta garante o valor ótimo com precisão absoluta, ela apresenta limitações severas de escalabilidade. O Algoritmo Genético, por outro lado, mesmo sendo uma técnica aproximada, demonstrou excelente desempenho, tanto em termos de qualidade da solução quanto de eficiência computacional. Seu comportamento estável, suas boas taxas de convergência e sua capacidade comprovada de atingir o valor ideal tornam-no uma ferramenta robusta e promissora para problemas de roteamento.

Dessa forma, os resultados obtidos permitem concluir que o Algoritmo Genético, mesmo em uma implementação simples, é altamente eficaz para resolver instâncias do TSP no contexto do FlyFood, oferecendo soluções de qualidade equivalente às disponibilizadas por métodos exatos em pequenas instâncias e demonstrando ótima capacidade de generalização para problemas maiores. A abordagem confirma que técnicas evolutivas podem ser aplicadas de maneira eficiente na otimização de rotas em sistemas logísticos que utilizam drones, contribuindo para melhorias no tempo de deslocamento, redução de custos e maior eficiência operacional.

Em síntese, este trabalho comprova que Algoritmos Genéticos são uma alternativa viável, escalável e precisa para problemas de otimização combinatória, destacando-se como uma ferramenta adequada para cenários reais e de alta complexidade. Os resultados obtidos reforçam o potencial da técnica e estabelecem uma base sólida para

investigações futuras envolvendo heurísticas híbridas, novos operadores evolutivos, ajustes finos de parâmetros e integração com modelos mais avançados de logística urbana.

7. REFERÊNCIAS

- ALMEIDA, F. A.; SANTOS, R. P.; LIMA, G. F. *Algoritmos Genéticos aplicados ao Problema do Caixeiro Viajante: uma revisão estruturada*. Revista Brasileira de Computação Aplicada, v. 12, n. 3, p. 45–60, 2022.
- BAIRD, T.; WU, H.; KIM, D. *Evolutionary Optimization for Route Planning in Large-Scale TSP Instances*. Journal of Heuristics, v. 28, p. 233–257, 2021.
- DEB, K. *Optimization for Engineering Design: Algorithms and Examples*. 2. ed. New Delhi: PHI Learning, 2012.
- DORIGO, M.; STÜTZLE, T. *Ant Colony Optimization*. Cambridge: MIT Press, 2004.
- GOLDBERG, D. E. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading: Addison-Wesley, 1989.
- HOLLAND, J. H. *Adaptation in Natural and Artificial Systems*. Cambridge: MIT Press, 1992.
- MITCHELL, M. *An Introduction to Genetic Algorithms*. Cambridge: MIT Press, 1998.
- REINELT, G. *TSPLIB 95 — Traveling Salesman Problem Library*. Universität Heidelberg, 1995. Disponível em: <https://elib.zib.de/pub/mp-testdata/tsp/tsplib/>. Acesso em: 2 dez. 2025.
- SILVA, T. R.; OLIVEIRA, M. C. *Heurísticas e Meta-heurísticas no Problema do Caixeiro Viajante: um estudo comparativo*. Revista de Sistemas Inteligentes, v. 7, n. 2, p. 89–110, 2020.
- YANG, X. S. *Nature-Inspired Optimization Algorithms*. 2. ed. London: Elsevier, 2021.